Spec-Driven Development

with spec-kit

Aaron Van Bokhoven
Vibecon 2026

github.com/github/spec-kit

Vibe coding works... until it doesn't

Great for prototypes, spikes, and small changes. But the moment a feature involves more than one session, one person, or one day - you lose visibility.

Why was it built that way	Decisions live in a chat session no one else can see.
How to hand it off	If you didn't write it, there's no trail of intent to follow.
Where to roll back	If something breaks at step 15, you're starting over.

Free-soloing a big feature — no fall protection, no fallback, if you fail you die.

The real problem: no visibility

Without structure

"Build me a login page"

→ Agent generates 30 files

→ Decides to build a new library

→ Why this ORM? Why this pattern?

→ Agent runs out of context

With spec-driven approach

Spec defines what & why

Plan defines how & tech stack

Tasks break it into atomic steps

Human reviews at each step

Full visibility: intent → code

What is a spec?

A structured document, written in plain language, that describes how your software should behave — and serves as the primary guide for your AI coding agent.

What is spec-driven development?

Instead of coding first and writing docs later, you start with a spec. It becomes the contract for how your code should behave and the source of truth your AI agent uses to generate, test, and validate code.

Why SDD works so well with LLMs

Natural Language	LLMs understand natural language. Specs are written in natural language. No translation layer needed.
Context Bloat	LLMs handle 10 files well. 20+ and coherence breaks. SDD forces atomic, scoped tasks.
Persistent memory	The spec is memory across sessions. The agent re-reads it every time — no context loss.
Checkpoints	Human reviews at each step. Catch drift early, not after 15 files are changed.

What is spec-kit?

Open source toolkit by GitHub (Den Delimarsky & John Lam).

The problem isn't the agent's ability — it's our approach. We treat agents like search engines when we should treat them like pair programmers who need clear instructions.

It's still Claude Code. It provides rails, not a replacement.

1. **A process** — Constitution → spec → plan → tasks → implement.
2. **Templates** — Opinionated templates that guide the agent toward consistency.
3. **Hooks** — Ensures context like your constitution is always loaded.

Indoor climbing with color-coded routes. You still have to climb.

The spec-kit workflow

1 Specify

You describe what you're building and why.

The agent generates a detailed spec focused on user journeys, experiences, and what success looks like — not technical details.

```
/speckit.specify Build "Send Log", a climbing send tracker.
```

→

2 Plan

You provide your desired stack, architecture, and constraints.

The agent generates a comprehensive technical plan that respects those decisions.

```
/speckit.plan Use TypeScript and Next.js with App Router. Tailwind CSS for styling with a dark theme.
```

→

3 Tasks

The agent breaks the spec and plan into small, reviewable chunks.

Each task should be something you can implement and test in isolation.

```
/speckit.tasks
```

→

4 Implement

The agent tackles tasks one by one.

Instead of reviewing thousand-line code dumps, you review focused changes tied to specific tasks.

```
/speckit.implement
```

The spec-kit workflow

1 → 2 → 3 → 4

Specify

You describe what you're building and why.

The agent generates a detailed spec focused on user journeys, experiences, and what success looks like — not technical details.

```
/speckit.specify Build "Send Log", a climbing send tracker.
```

Plan

You provide your desired stack, architecture, and constraints.

The agent generates a comprehensive technical plan that respects those decisions.

```
/speckit.plan Use TypeScript and Next.js with App Router. Tailwind CSS for styling with a dark theme.
```

Tasks

The agent breaks the spec and plan into small, reviewable chunks.

Each task should be something you can implement and test in isolation.

```
/speckit.tasks
```

Implement

The agent tackles tasks one by one.

Instead of reviewing thousand-line code dumps, you review focused changes tied to specific tasks.

```
/speckit.implement
```

The spec-kit workflow

1 Specify

You describe what you're building and why.

The agent generates a detailed spec focused on user journeys, experiences, and what success looks like — not technical details.

```
/speckit.specify Build "Send Log", a climbing send tracker.
```

→

2 Plan

You provide your desired stack, architecture, and constraints.

The agent generates a comprehensive technical plan that respects those decisions.

```
/speckit.plan Use TypeScript and Next.js with App Router. Tailwind CSS for styling with a dark theme.
```

→

3 Tasks

The agent breaks the spec and plan into small, reviewable chunks.

Each task should be something you can implement and test in isolation.

```
/speckit.tasks
```

→

4 Implement

The agent tackles tasks one by one.

Instead of reviewing thousand-line code dumps, you review focused changes tied to specific tasks.

```
/speckit.implement
```

The spec-kit workflow

1 Specify

You describe what you're building and why.

The agent generates a detailed spec focused on user journeys, experiences, and what success looks like — not technical details.

```
/speckit.specify Build "Send Log", a climbing send tracker.
```

→

2 Plan

You provide your desired stack, architecture, and constraints.

The agent generates a comprehensive technical plan that respects those decisions.

```
/speckit.plan Use TypeScript and Next.js with App Router. Tailwind CSS for styling with a dark theme.
```

→

3 Tasks

The agent breaks the spec and plan into small, reviewable chunks.

Each task should be something you can implement and test in isolation.

```
/speckit.tasks
```

→

4 Implement

The agent tackles tasks one by one.

Instead of reviewing thousand-line code dumps, you review focused changes tied to specific tasks.

```
/speckit.implement
```

The constitution is the secret weapon

`/speckit.constitution`

What goes in a constitution

- Your team's coding patterns
- Architecture conventions
- Testing standards & requirements
- Performance constraints
- Error handling approaches
- What NOT to do

Why it matters

Project-specific knowledge that normally live in a silo such as in code review comments, Slack threads, and past PRs.

The agent reads this before implementing anything — getting your standards, not generic best practices from training data.

Clip the bolt, don't free solo the feature

```
/speckit.implement Phase 4 only
```

Each completed phase is a natural commit point.

Phase 1	→	Phase 2	→	Phase 3	→	Phase 4
√ commit		√ commit		√ commit		✗ went wrong

Roll back to phase 3, not the whole feature.

- Git-friendly: each phase = one focused commit with a clear description.
- If the agent goes off the rails on phase 7, you haven't lost phase 1–6.
- You can retry a single phase with different instructions without starting over.

Artifacts are documented

spec.md, plan.md, and research.md are standalone documents — not trapped in a chat session or in spec-kit.

Get a second opinion

- Feed the spec to another agent
- Ask a different LLM to review the plan

Generate downstream docs

- ADRs from the plan + research
- PRDs from the spec
- READMEs from the full artifact set
- Onboarding docs from the constitution

The docs you always meant to write.

what spec-kit looks like

An example of a spec-kit coded project

---

constitution.md spec.md plan.md tasks.md the code

What I've Learned

Being honest about the limits

Small bug fixes are overkill — "like using a sledgehammer to crack a nut" [0]

It generates a lot of markdown.
For small features, it can feel like overhead.
spec-first vs spec-anchored [0]

Agents are still unpredictable. SDD gives you structure, not guarantees.

LLMs and tooling are getting better.

But you're not locked in

Constitution, spec, plan, tasks

— These are just markdown files.

Works with any agent

— Claude Code, Codex, Cursor, Copilot.

Spec Driven Development

— You can apply the patterns without spec-kit

[0] Böckeler - martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Now it's your turn

Phase 2: Hands-on

1.	`/speckit.specify`	Describe your one-line feature. Focus on what and why.
2.	`/speckit.plan`	Add your tech stack and architecture decisions.
3.	`/speckit.implement`	Execute 1–2 tasks. Experience the process.

github.com/aaronvb/send-log